并行程序设计与算法实验

4-Pthreads 并行方程求解及蒙特卡洛

提交格式说明

按照实验报告模板填写报告，需要提供源代码及代码描述至 https://easyai.sysu.edu.cn/course/13。实验报告模板使用 PDF 格式，命名方式为“lab4_学号_姓名”。如有疑问，请发送邮件至 yuhw8@mail2.sysu.edu.cn 余华威询问细节。

1. 一元二次方程求解

使用 Pthread 编写多线程程序，求解一元二次方程组的根，结合数据及任务之间的依赖关系，及实验计时，分析其性能。

一元二次方程：为包含一个未知项，且未知项最高次数为二的整式方程式，常写作$ax^2 + bx + c = 0$，其中x为未知项，a, b, c为三个常数。

一元二次方程的解：一元二次方程的解可由求根公式给出：

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

输入：a, b, c三个浮点数，其的取值范围均为[-100, 100]

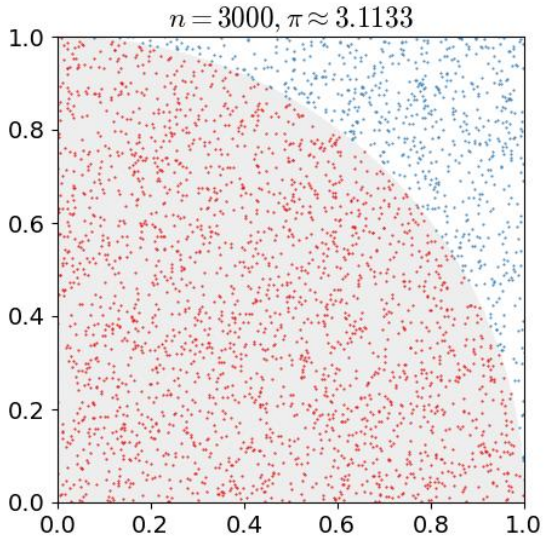
问题描述：使用求根公式并行求解一元二次方程$ax^2 + bx + c = 0$。

输出：方程的解x_1, x_2，及求解所消耗的时间t。

要求：使用 Pthreads 编写多线程程序，根据求根公式求解一元二次方程。求根公式的中间值由不同线程计算，并使用条件变量识别何时线程完成了所需计算。讨论其并行性能。

2. 蒙特卡洛方法求π的近似值

基于 Pthreads 编写多线程程序，使用蒙特卡洛方法求圆周率π近似值。



蒙特卡洛方法与圆周率近似：蒙特卡洛方法是一种基于随机采样的数值计算方法，通过模拟随机时间的发生，来解决各类数学、物理和工程上的问题，尤其是直接解析解决困难或无法求解的问题。其基本思想是：当问题的确切解析解难以获得时，可以通过随机采样的方式，生成大量的模拟数据，然后利用这些数据的统计特性来近似求解问题。在计算圆周率π值时，可以随机地将点撒在一个正方形内。当点足够多时（见上图），总采样点数量与落在内切圆内采样点数量的比例将趋近于$\pi/4$，可据此来估计π的值。

输入：整数n，取值范围为[1024, 65536]

问题描述：随机生成正方形内的n个采样点，并据此估算π的值。

输出：总点数n，落在内切圆内点数m，估算的π值，及消耗的时间t。

要求：基于 Pthreads 编写多线程程序，使用蒙特卡洛方法求圆周率π近似值。讨论程序并行性能。

附录 1：环境配置及软件安装

本实验提供两种环境配置方式，你可以根据个人需求选择：

方式 1：本地虚拟机环境安装

1.下载安装虚拟机软件（推荐 VMware 或 VirtualBox）

a) VMware
https://www.vmware.com/products/desktop-hypervisor/workstation-and-fusion

b) VirtualBox
https://www.virtualbox.org/wiki/Downloads

2. 下载 Ubuntu 22.04 镜像文件
http://releases.ubuntu.com/22.04/ （官方）
https://mirrors.sustech.edu.cn/ubuntu-releases/22.04/ （国内）

3. 在 VMware 或 VirtualBox 中创建虚拟机实例，并调整其资源，如 CPU 核、内存、硬盘容量等。

4. 在虚拟机实例中的虚拟光驱中加载 Ubuntu 镜像文件(.iso 文件)，并安装 Ubuntu 操作系统。

5. 软件安装

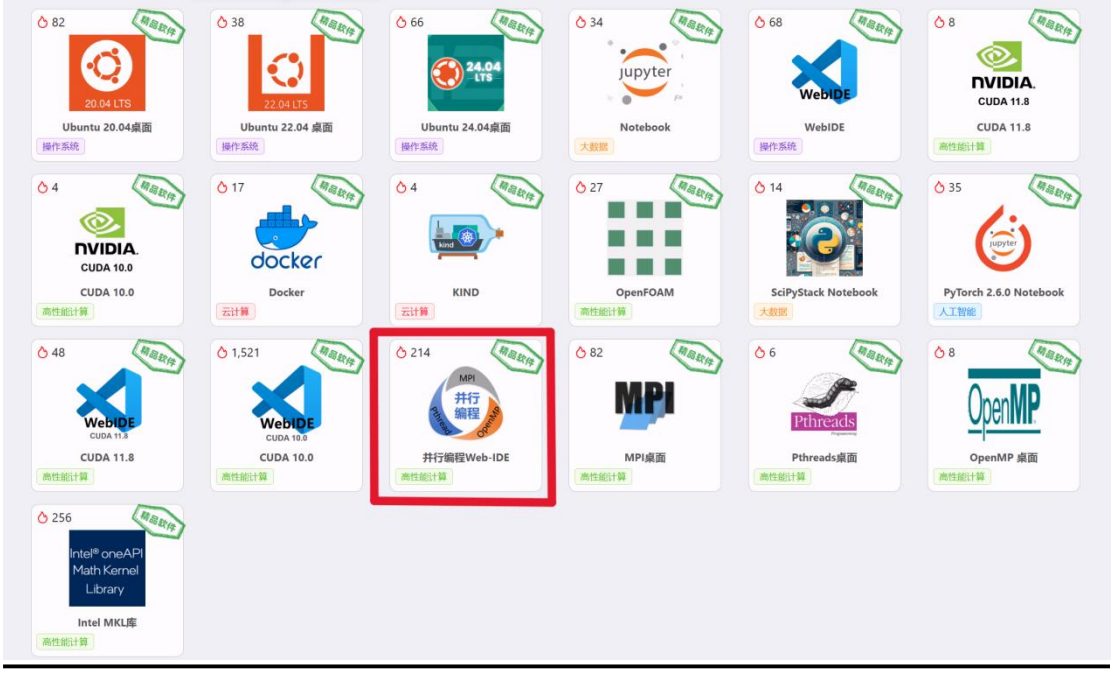
Pthreads 是 Linux/glibc 自带线程库，不需要单独安装开发包：

```
sudo apt-get update
sudo apt-get install build-essential git
```

方式 2：使用智算习堂实验环境

使用“并行编程 Web-IDE”容器，其中预装了 GCC、Make 等 C 语言编译工具。Pthreads 为 Linux 系统自带线程库，编译时添加 -pthread 选项即可。

（https://easyai.sysu.edu.cn/software?id=22）



可通过以下指令验证编译工具

- 检查编译器：gcc --version
- 检查构建工具：make --version
- 检查 Pthread 可用性：

```
cat > check_pthread.c << 'EOF'
#include <pthread.h>
int main() { return 0; }
EOF

gcc -std=c11 -pthread check_pthread.c -o check_pthread

./check_pthread && echo Pthreads 环境可用
```